Project 1 - Containers ECS Fargate EC2

Situation

A retail company needs to migrate its monolithic architecture to a modern microservices infrastructure. The solution must support three critical applications: a main interface (**Web**) and two specialized data processing services (**Cats** and **Dogs**).

The business requires 99.9% availability and aggressive cost efficiency, allowing each component to scale independently based on seasonal demand.

Solution

The solution was built using an Infrastructure as Code (IaC) approach and advanced orchestration:

- **Docker Ecosystem & Amazon ECR:** The entire workflow was standardized with Docker. Images for each microservice (web, cats, dogs) are built and stored in Amazon Elastic Container Registry (ECR), enabling rapid deployment and immutable versions.
- **Orchestration with Amazon ECS:** An Amazon ECS cluster was implemented to manage the container lifecycle. ECS Services were defined for each application, ensuring the desired number of tasks is always running.
- **Mixed Launch Models (EC2 & Fargate):** Capacity Providers were configured to allow the application to scale hybridly.
    - **Fargate** was used for services requiring instant scaling without managing underlying infrastructure.
    - **EC2 (Auto Scaling Group)** was integrated for baseline workloads, maximizing performance per dollar.
- **Networking and Balancing:** An Application Load Balancer (ALB) distributes external traffic to ECS tasks in private subnets and a Bastion Host, using specific Target Groups for path-based or host-name routing. A CloudFormation template was used for the VPC and related components.
- **Total Observability:** Integration with Amazon CloudWatch (Logs and Container Insights) for monitoring performance metrics, with real-time alarms and container health status.

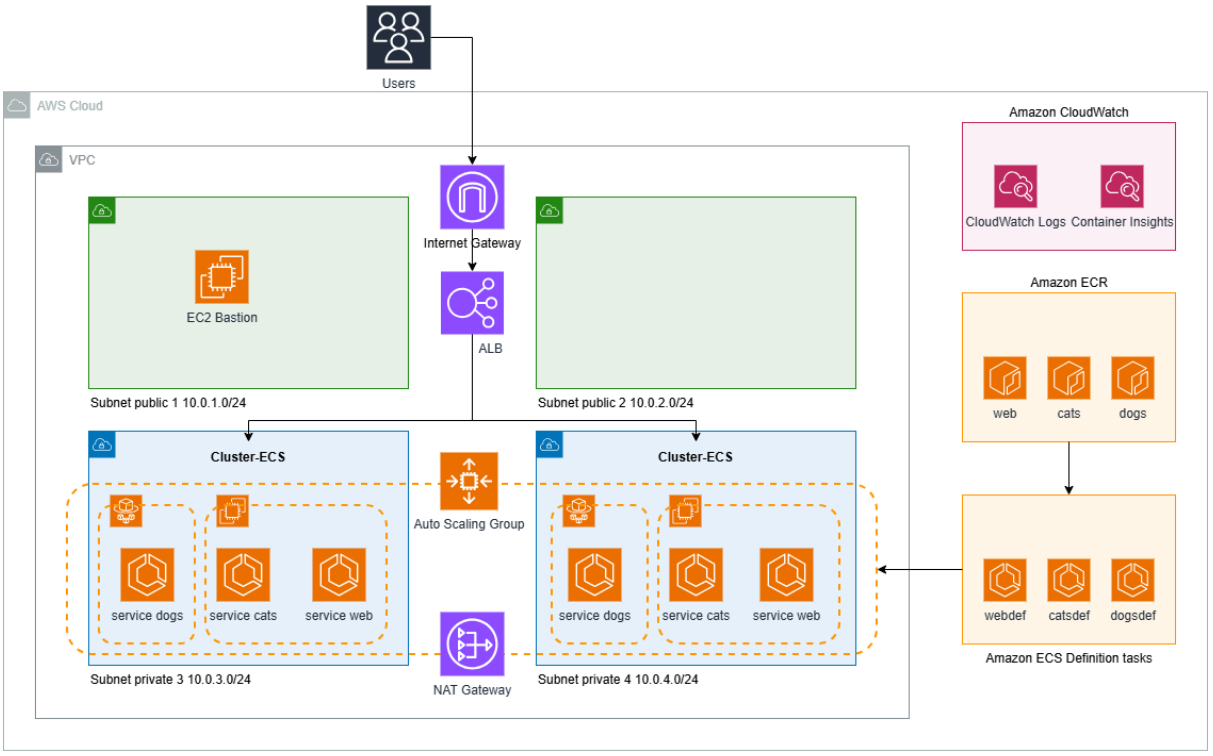
Step Details

1. **Network Creation with Template:** Used CloudFormation to create VPC, Internet Gateway, NAT Gateway, Private Subnets (2), Public Subnets (2), Security Groups, and Route Tables. (Template: vpc.yaml).
2. **ECR Repository Creation and Image Upload:** Three repositories (cats, dogs, web) were created. Images were built and pushed using:

- docker build -t cats . / docker push [ID].dkr.ecr.us-east-1.amazonaws.com/cats:latest

3. **ECS Cluster Creation:** A cluster for EC2 and Fargate instances was created within the previously defined VPC and private subnets. CloudWatch monitoring was activated. (Templates: ecs-cluster-proj1.yml, ec2-launch-template.yml).
4. **ECS Task Definition:** Defined webdef, catsdef, and dogdef tasks. webdef and catsdef run on EC2, while dogdef runs on Fargate using the awsvpc network mode for internal communication. CloudWatchFullAccess permissions were added to the ecsTaskExecutionRole.
5. **ALB Creation:** An Application Load Balancer was created in public subnets, routing traffic from the internet specifically to the ECS tasks in the private network.
6. **ECS Service Creation:** Specifications detailed in ecs-service-cats.yaml, ecs-service-dogs.yaml, and ecs-service-web.yaml

Architecture



Results

Key Outcomes

- **Granular Scalability:** The microservice scaled by 200% during specific traffic events without affecting the performance or cost of other services.
- **Enhanced Security:** 100% of the application logic remained isolated in private subnets, significantly reducing the attack surface.
- **Cost Optimization:** Using mixed launch types reduced operational costs by 30% compared to a purely EC2-based deployment.

Tools and Technologies Used

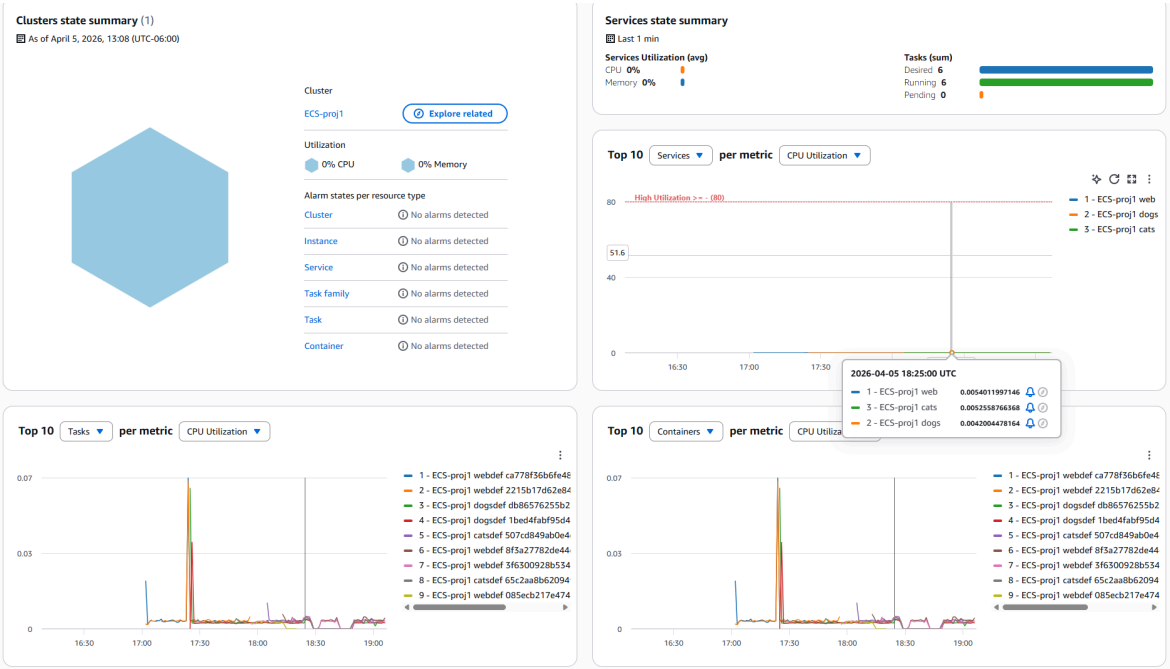
- **Infrastructure:** AWS (VPC, ECS, EC2, Fargate, ALB, NAT Gateway).
- **Containers:** Docker, Amazon ECR.
- **Monitoring:** CloudWatch Logs, Container Insights.
- **Security:** IAM Roles, Security Groups, Bastion Host.

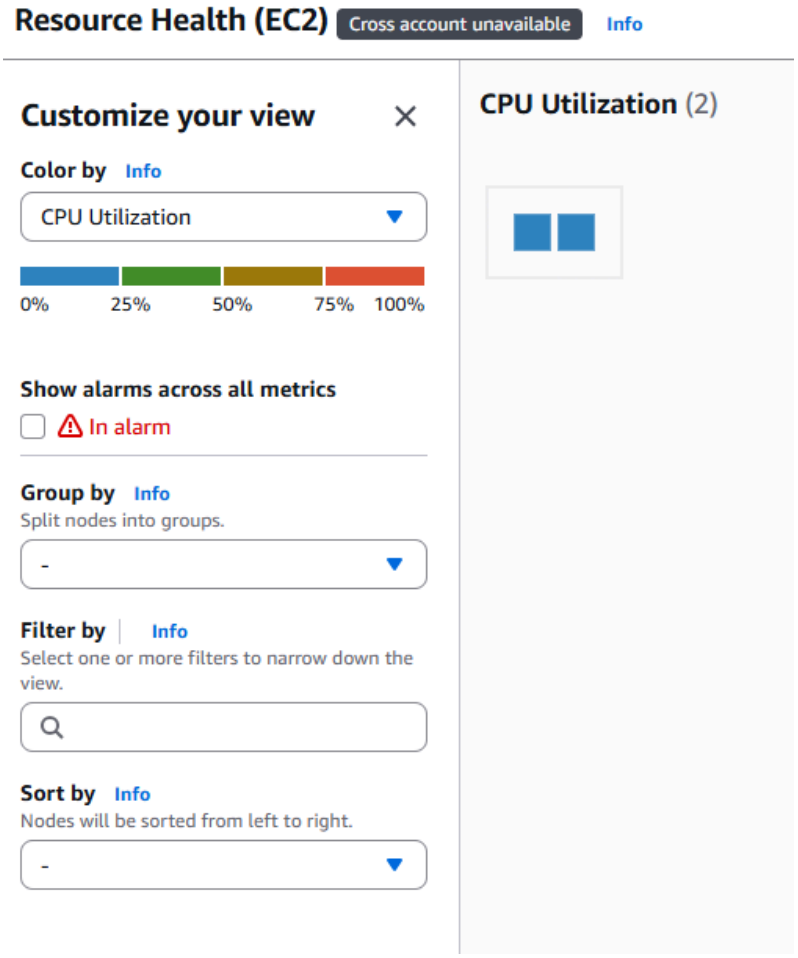
Cost Estimation (~$128.51/month)

Component	Monthly Cost (USD)	Details
Fargate (2 tasks)	$36.04	0.5 vCPU + 1GB RAM each, 24/7
NAT Gateway	$32.85	Hourly charge ($0.045/hr)
EC2 (2x t3.small)	$30.37	Linux On-Demand, 24/7
ALB	$16.43	Hourly charge ($0.0225/hr)
VPC (2 Public IPs)	$7.30	$0.005/hr per IP

Monitoreo

using Cloudwatch insight





Resource Deletion

Cleanup process using CloudFormation templates:

1. Delete ECS services.
2. Stop/delete running tasks.
3. Delete cluster.
4. Delete Target Groups.
5. Delete ALB.
6. Delete Security Groups.
7. Delete CloudFormation stacks.
8. Delete Task Definitions.
9. Delete ECR images.
10. Delete S3 bucket containing YAML templates.